

8.Hafta - Lojistik Regresyon Modeli

April 23, 2025

1 8.Hafta - Lojistik Regresyon Modeli

5.1. Lojistik Regresyon modelinin uygulanması ve hiperparametrelerinin açıklanması

5.2. Modelin eğitilmesi

- Özellik ölçekleme yöntemlerinin uygulanması

5.3. Model sonuçlarının değerlendirilmesi

1.0.1 5.1. Lojistik Regresyon modelinin uygulanması ve hiperparametrelerinin açıklanması

Logistic regression (Lojistik regresyon), bağımlı bir değişken ile bir veya daha fazla bağımsız değişken arasındaki ilişkiyi modellemek için kullanılan bir istatistiksel yöntemdir. Ancak doğrusal regresyondan farkı, bağımlı değişkenin sınıflandırma problemiyle ilgilenmesidir. Yani bağımlı değişken genellikle kategorik olur ve değerleri belirli sınıflar arasında değişir (örneğin, 0 ve 1 gibi).

```
[108]: # gerekli kütüphaneleri import edelim
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
```

```
[109]: # veri setimizi okuyalım
df = pd.read_csv("cleaned_df_data.csv")
```

```
[110]: df
```

```
[110]:
```

	age	job	marital	education	default	housing	loan	\
0	58	management	married	tertiary	0	1	0	
1	44	technician	single	secondary	0	1	0	
2	33	entrepreneur	married	secondary	0	1	1	
3	47	blue-collar	married	unknown	0	1	0	
4	33	unknown	single	unknown	0	0	0	
...	...	...	...	...	...	...	...	
45206	51	technician	married	tertiary	0	0	0	

45207	71	retired	divorced	primary	0	0	0
45208	72	retired	married	secondary	0	0	0
45209	57	blue-collar	married	secondary	0	0	0
45210	37	entrepreneur	married	secondary	0	0	0

	contact	month	campaign	previous	poutcome	y	balance_level	\
0	unknown	may	1	0	unknown	0	2	
1	unknown	may	1	0	unknown	0	1	
2	unknown	may	1	0	unknown	0	1	
3	unknown	may	1	0	unknown	0	1	
4	unknown	may	1	0	unknown	0	1	
...	...	...	...	...	...	...	...	
45206	cellular	nov	3	0	unknown	1	1	
45207	cellular	nov	2	0	unknown	1	1	
45208	cellular	nov	5	3	success	1	2	
45209	telephone	nov	4	0	unknown	0	1	
45210	cellular	nov	2	11	other	0	2	

	has_any_credit
0	1
1	1
2	1
3	1
4	0
...	...
45206	0
45207	0
45208	0
45209	0
45210	0

[45211 rows x 15 columns]

```
[111]: df = pd.get_dummies(df,
                           columns=["job", "marital", "education", "contact", "month",
                                   ↪ "poutcome"],
                           drop_first=True)
```

```
[112]: # burada kaç veri var
df.shape
```

```
[112]: (45211, 41)
```

```
[113]: df
```

```
[113]:
```

	age	default	housing	loan	campaign	previous	y	balance_level	\
0	58	0	1	0	1	0	0	2	

1	44	0	1	0	1	0	0	1
2	33	0	1	1	1	0	0	1
3	47	0	1	0	1	0	0	1
4	33	0	0	0	1	0	0	1
...	...	...	...	...	...	...	...	...
45206	51	0	0	0	3	0	1	1
45207	71	0	0	0	2	0	1	1
45208	72	0	0	0	5	3	1	2
45209	57	0	0	0	4	0	0	1
45210	37	0	0	0	2	11	0	2

	has_any_credit	job_blue-collar	...	month_jul	month_jun	month_mar	\
0	1	0	...	0	0	0	
1	1	0	...	0	0	0	
2	1	0	...	0	0	0	
3	1	1	...	0	0	0	
4	0	0	...	0	0	0	
...	...	...	...	...	...	...	
45206	0	0	...	0	0	0	
45207	0	0	...	0	0	0	
45208	0	0	...	0	0	0	
45209	0	1	...	0	0	0	
45210	0	0	...	0	0	0	

	month_may	month_nov	month_oct	month_sep	poutcome_other	\
0	1	0	0	0	0	
1	1	0	0	0	0	
2	1	0	0	0	0	
3	1	0	0	0	0	
4	1	0	0	0	0	
...	...	...	...	...	...	
45206	0	1	0	0	0	
45207	0	1	0	0	0	
45208	0	1	0	0	0	
45209	0	1	0	0	0	
45210	0	1	0	0	1	

	poutcome_success	poutcome_unknown
0	0	1
1	0	1
2	0	1
3	0	1
4	0	1
...	...	...
45206	0	1
45207	0	1
45208	1	0

```

45209          0          1
45210          0          0

```

```
[45211 rows x 41 columns]
```

```
[114]: df.isna().any().sum()
```

```
[114]: 0
```

```
[115]: df
```

```

[115]:      age  default  housing  loan  campaign  previous  y  balance_level  \
0      58         0         1     0         1         0  0         2
1      44         0         1     0         1         0  0         1
2      33         0         1     1         1         0  0         1
3      47         0         1     0         1         0  0         1
4      33         0         0     0         1         0  0         1
...
45206   51         0         0     0         3         0  1         1
45207   71         0         0     0         2         0  1         1
45208   72         0         0     0         5         3  1         2
45209   57         0         0     0         4         0  0         1
45210   37         0         0     0         2        11  0         2

      has_any_credit  job_blue-collar  ...  month_jul  month_jun  month_mar  \
0                  1                0  ...         0         0         0
1                  1                0  ...         0         0         0
2                  1                0  ...         0         0         0
3                  1                1  ...         0         0         0
4                  0                0  ...         0         0         0
...
45206              0                0  ...         0         0         0
45207              0                0  ...         0         0         0
45208              0                0  ...         0         0         0
45209              0                1  ...         0         0         0
45210              0                0  ...         0         0         0

      month_may  month_nov  month_oct  month_sep  poutcome_other  \
0              1          0          0          0              0
1              1          0          0          0              0
2              1          0          0          0              0
3              1          0          0          0              0
4              1          0          0          0              0
...
45206          0          1          0          0              0
45207          0          1          0          0              0
45208          0          1          0          0              0

```

45209	0	1	0	0	0
45210	0	1	0	0	1

	poutcome_success	poutcome_unknown
0	0	1
1	0	1
2	0	1
3	0	1
4	0	1
...	...	...
45206	0	1
45207	0	1
45208	1	0
45209	0	1
45210	0	0

[45211 rows x 41 columns]

45211 veriden 40000 tanesi ile öğretilim, 5211 tanesi ile test edelim.

en sık yüzdelik ayırma olarak %70-%30 veya %75-%25 olarak milyonlu satırlara çıktıkça %90'a yakın train olarak verilebilir.

```
[116]: # y sınıf oranlarını görelim
print(df["y"].value_counts(normalize=True))
```

```
0    0.883015
1    0.116985
Name: y, dtype: float64
```

Veri setimiz dengesiz bir yapıya sahip. Veri dengeleme yöntemlerinden class_weight='balanced' parametresi ile bu durum incelenecektir.

1.0.2 5.2. Modelin eğitilmesi

- Özellik ölçekleme yöntemlerinin uygulanması

```
[117]: # 1. y tahmin etmek istediğimiz yani bağımlı değişken, x = feature'lar yani
↳ bağımsız değişkenler
```

```
y = df["y"]
X = df.drop("y", axis=1)
```

```
[118]: # 2. veriyi train ve test olarak ayıralım
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.70,
↳ random_state=42)
```

```
# train_size= %77 oranında, random_state= satırları rastgele verirse karışık
↳ öğrenmesi için daha iyi olur o yüzden random_state kullanılır. karşıdaki
↳ kişiyle aynı olması için aynı değer verilir.
```

StandardScaler, aşırı büyük veya küçük değerlerin etkisini azaltır.

Eğer modelleme için verilerin aynı ölçekte olması gerekiyorsa bu yöntem kullanılmalıdır.

```
[119]: # sklearn'den StandardScaler'ı import edelim

from sklearn.preprocessing import StandardScaler
```

```
[120]: # 3. StandardScaler ile ölçekle

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train) # sadece train seti üzerinde fit
X_test_scaled = scaler.transform(X_test)      # test setini sadece transform
↳ et
```

```
[121]: # 4. Logistic Regression modelini eğit

log_model = LogisticRegression(max_iter=1000, class_weight='balanced') #
↳ modeli çağırıyoruz
log_model.fit(X_train_scaled, y_train) # modeli eğitiyoruz
↳ (öğrenme tamamlanacak)
```

```
[121]: LogisticRegression(class_weight='balanced', max_iter=1000)
```

max_iter=1000 Parametresi Ne İşe Yarar?

LogisticRegression, modeli eğitirken optimizasyon algoritması kullanır (varsayılan olarak 'lbfgs' algoritması).

Bu algoritma, en iyi model parametrelerini bulmak için iteratif (yinelemeli) olarak çalışır.

max_iter, bu maksimum tekrar (iteration) sayısını belirler.

Yani:

Model, çözüm (optimum ağırlıklar) bulana kadar denemeye devam eder ama en fazla 1000 adım ilerleyebilir.

Eğer 1000 adım sonunda hâlâ çözüm bulunamadıysa eğitim erken durur ve ConvergenceWarning (yakınsama uyarısı) verir.

Neden Değeri Arttırma İhtiyacı Duyarız? - Veri seti çok büyükse, - Öznitelik sayısı fazla ise, - Veya ölçekleme yapılmamışsa, model yakınsamakta zorlanabilir.

max_iter=1000, modelin öğrenme sürecinde kaç kez deneme yapabileceğini sınırlar. Bu değer çok düşükse model yeterince öğrenemeyebilir, çok yüksekse ise eğitim süresi uzar ama genelde sorun olmaz.

`class_weight='balanced'` ne yapar?

Dengesiz veri setini düzenlemek için kullanılan parametrelerden birisidir. Bu parametre, her sınıfa ters orantılı ağırlıklar verir. Yani:

- Az olan sınıfa daha yüksek ağırlık,
- Çok olan sınıfa daha düşük ağırlık verilir.

Bu sayede model, her iki sınıfa da eşit önem vermeye çalışır.

1.0.3 5.3. Model sonuçlarının değerlendirilmesi

```
[122]: # 5. Test seti tahminleri
y_pred = log_model.predict(X_test_scaled)

# 6. Performans değerlendirmesi
print("Logistic Regression Sınıflandırma Raporu:")
print(classification_report(y_test, y_pred)) # Kapsamlı rapor
```

Logistic Regression Sınıflandırma Raporu:

	precision	recall	f1-score	support
0	0.94	0.77	0.84	11966
1	0.26	0.62	0.37	1598
accuracy			0.75	13564
macro avg	0.60	0.69	0.60	13564
weighted avg	0.86	0.75	0.79	13564

```
[123]: # 7. Model doğruluğunu hesapla

from sklearn.metrics import accuracy_score, precision_score, recall_score, \
    f1_score, confusion_matrix

accuracy = accuracy_score(y_test, y_pred)
print(f"Logistic Regression Doğruluk (Accuracy): {accuracy:.2f}")
```

Logistic Regression Doğruluk (Accuracy): 0.75

1.0.4 model -> %75 oranında doğru bilmiş.

```
[124]: # 8. Performans metriklerini hesapla
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```
print(f"Logistic Regression Kesinlik (Precision): {precision:.2f}")
print(f"Logistic Regression Duyarlılık (Recall): {recall:.2f}")
print(f"Logistic Regression F1-Skoru: {f1:.2f}")
```

Logistic Regression Kesinlik (Precision): 0.26

Logistic Regression Duyarlılık (Recall): 0.62

Logistic Regression F1-Skoru: 0.37

1. Accuracy (Doğruluk):

- Modelin doğru sınıflandırdığı örneklerin toplam örneklere oranıdır.
- Değer: 0.75 (yani %75 doğruluk) bu, modelin toplamda doğru sınıflandırmalarının oranının oldukça yüksek olduğunu gösteriyor. Ancak doğruluk, dengesiz veri setlerinde yanıltıcı olabilir, çünkü nadir sınıfların doğru tahmin edilmesini göz ardı edebilir.

2. Precision (Kesinlik):

- Modelin pozitif olarak sınıflandırdığı örneklerin gerçekte pozitif olma oranıdır. Yani modelin “pozitif” tahminlerinde ne kadar doğru olduğu.
- Değer: 0.26 (yani %26 kesinlik) bu, modelin pozitif tahminlerinde genellikle doğru olduğunu, ancak yanlış pozitiflerin de var olduğunu gösteriyor. Precision’ın daha yüksek olması, yanlış pozitiflerin sayısının düşük olduğu anlamına gelir.

3. Recall (Duyarlılık):

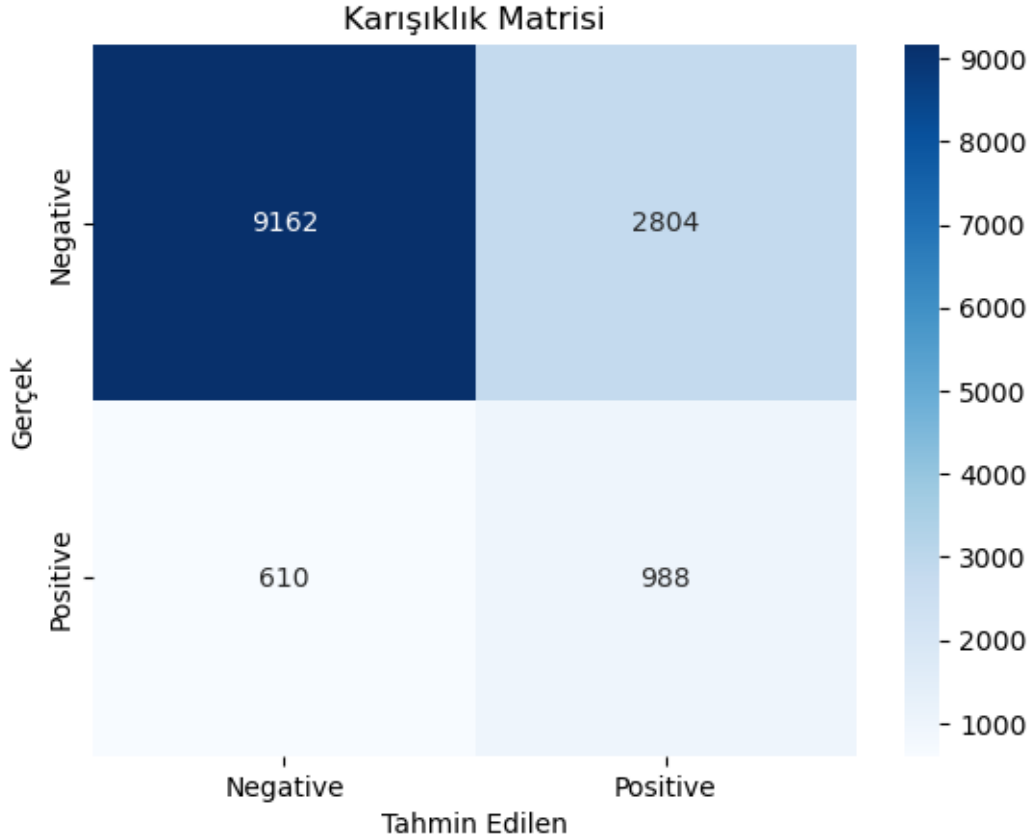
- Gerçek pozitiflerin model tarafından doğru tahmin edilme oranıdır. Yani modelin tüm gerçek pozitifleri ne kadar iyi bulduğunu gösterir.
- Değer: 0.62 (yani %62 duyarlılık) bu, modelin gerçek pozitif örneklerin çok küçük bir kısmını doğru tahmin edebildiğini gösterir. Yüksek duyarlılık, modelin nadir sınıfları doğru şekilde tanıyabilmesi açısından önemlidir.

4. F1 Score (F1 Skoru):

- Precision ve Recall’un harmonik ortalamasıdır ve bu iki metrik arasındaki dengeyi gösterir. Bu, genellikle sınıflandırma modelinin genel performansını değerlendirmek için daha dengeli bir ölçümdür, çünkü hem yanlış pozitifleri (precision) hem de yanlış negatifleri (recall) göz önünde bulundurur.
- Değer: 0.37 (yani %37 F1 skoru) bu, modelin genel anlamda hem yanlış pozitif hem de yanlış negatiflerle ilgili büyük zorluklar yaşadığını ve modelin dengesiz olduğunu gösteriyor. Yüksek F1 skoru daha iyi bir performansı gösterir.

1.0.5 Karışıklık Matrisi

```
[125]: # 9. Karışıklık matrisi
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Negative', 'Positive'], yticklabels=['Negative', 'Positive'])
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek')
plt.title('Karışıklık Matrisi')
plt.show()
```

confusion_matrix: Gerçek (y_{test}) ve tahmin edilen (y_{pred}) değerleri karşılaştırarak 2x2'lik bir matris oluşturur.

Bu matriste yer alan değerler:

- 9162 → Gerçek negatif, doğru tahmin edilmiş (True Negative - TN)
- 2804 → Gerçek negatif, yanlış pozitif tahmin edilmiş (False Positive - FP)
- 610 → Gerçek pozitif, yanlış negatif tahmin edilmiş (False Negative - FN)
- 988 → Gerçek pozitif, doğru tahmin edilmiş (True Positive - TP)

`sns.heatmap()`: Matrisin ısı haritasını çizer.

`cm`: Görselleştirilecek karışıklık matrisi.

`annot=True`: Hücrelerin içine sayıları yazar.

`fmt='d'`: Sayılar tamsayı formatında gösterilsin.

`cmap='Blues'`: Renk skalası olarak mavi tonları kullanılır. Koyu renkler yüksek değerleri gösterir.

`xticklabels` ve `yticklabels`: X ve Y ekseninde hangi sınıfların olduğunu belirtir. Burada “Negative” ve “Positive”.

Genel Çıkarım:

- Model, negatif sınıfı çok başarılı bir şekilde tahmin ediyor (TN çok yüksek).
- Pozitif sınıfı bulma konusunda başarısız (TP az, FN çok). Yani model çoğu pozitif veriyi gözden kaçırıyor.
- Bu durum, dengesiz veri seti (class imbalance) olasılığını düşündürür: Negatif sınıf daha fazla olabilir.
- Böyle bir durumda ROC AUC, Precision-Recall curve, SMOTE gibi veri dengeleme veya farklı metrikler kullanmak gerekir.

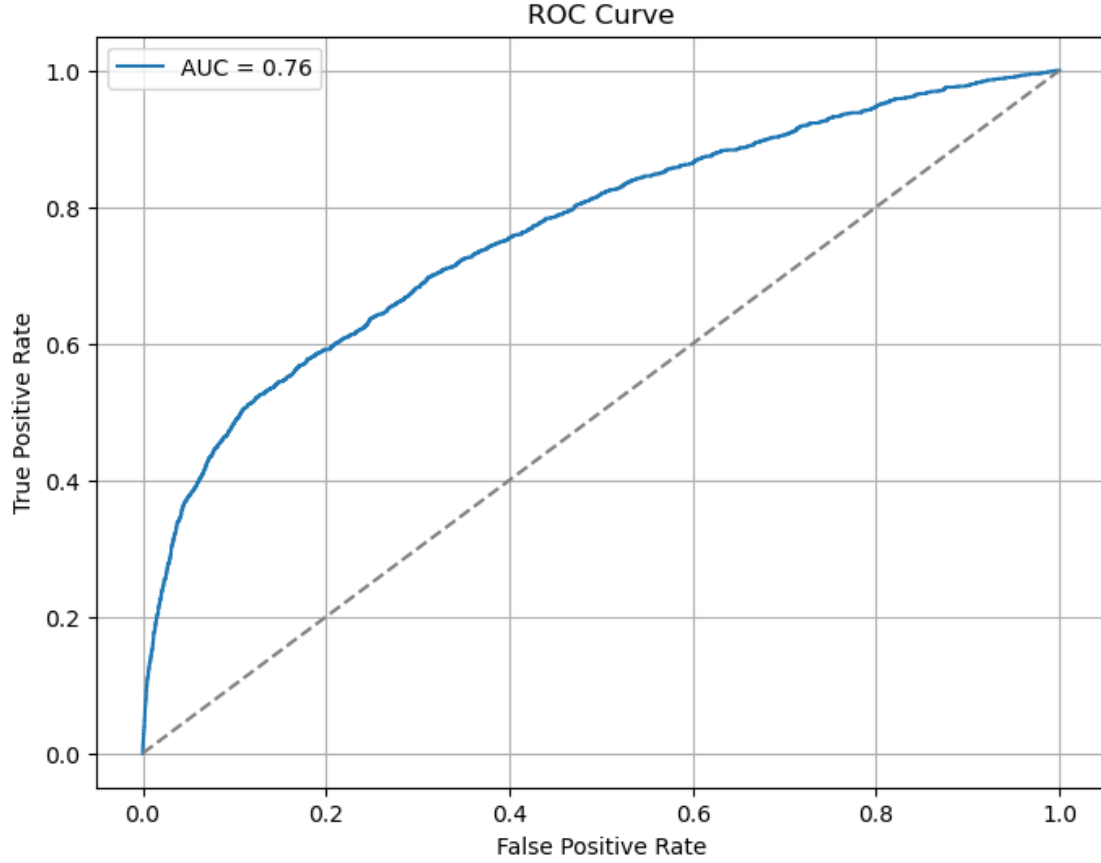
1.0.6 ROC Curve Eğrisi

```
[126]: from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as plt

# Pozitif sınıf için olasılık tahmini alalım
y_proba = log_model.predict_proba(X_test_scaled)[: , 1]

# ROC eğrisi için gerekli değerleri al
fpr, tpr, thresholds = roc_curve(y_test, y_proba)

# ROC eğrisini çiz
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc_score(y_test, y_proba):.2f}")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.grid(True)
plt.show()
```



ROC Curve

`roc_curve`: ROC eğrisi için gerekli olan 3 çıktıyı verir: FPR (False Positive Rate), TPR (True Positive Rate) ve eşik değerler.

`roc_auc_score`: AUC skorunu (alanı) hesaplar.

`matplotlib.pyplot`: Grafikleri çizmek için kullanılır.

`predict_proba()` fonksiyonu her sınıf için olasılık verir.

`[:, 1]` ifadesi, sadece pozitif sınıfa ait olasılıkları alır (1. sınıf = Positive).

Çünkü ROC eğrisi pozitif sınıf üzerinden değerlendirme yapar.

fpr (False Positive Rate): Yanlış pozitif oranı (negatif sınıfların pozitif olarak tahmin edilme oranı).

tpr (True Positive Rate): Doğru pozitif oranı (duyarlılık / recall).

thresholds: Olasılık eşik değerleri. Her eşikte fpr ve tpr yeniden hesaplanır.

ROC eğrisi çizilir.

X eksen: False Positive Rate

Y eksen: True Positive Rate

Label: AUC değeri hesaplanıp gösterilir. Bu görselde: $AUC = 0.76$

Bu çizgi, rastgele bir modelin performansını gösterir. $AUC = 0.5$ olan ideal olmayan bir model.

ROC eğrisinin bu çizginin üstünde olması, modelin anlamlı sınıflandırma yaptığı anlamına gelir.

AUC (Area Under Curve) = 0.76:

Modelin pozitif ve negatif sınıfları ayırt etme başarısını ölçer.

$AUC = 0.76 \rightarrow$ Model, rastgele tahminden oldukça daha iyi.

AUC 'nin ideal değeri 1.0'dır. Bu da hatasız ayırım anlamına gelir.

- 0.5 = Rastgele sınıflama.
- 0.7–0.8 arası: Kabul edilebilir bir model.
- 0.8–0.9: İyi bir model.
- 0.9: Harika model.

Bu ROC eğrisi sayesinde anlıyoruz ki modelin pozitif sınıfı ayırt etme kabiliyeti orta düzeyde ($AUC = 0.76$).

Bu da dengesiz veri setlerinde Accuracy'nin yanıltıcı olabileceğini bir kez daha gösteriyor.

1.0.7 Özellik Analizi

```
[127]: coeff_df = pd.DataFrame({
        "Feature": X.columns,
        "Coefficient": log_model.coef_[0]
    }).sort_values(by="Coefficient", key=abs, ascending=False)

    print(coeff_df.head(10)) # En etkili 10 özellik
```

	Feature	Coefficient
25	contact_unknown	-0.543851
38	poutcome_success	0.418461
26	month_aug	-0.291326
34	month_nov	-0.255470
33	month_may	-0.246291
30	month_jul	-0.240626
4	campaign	-0.224092
29	month_jan	-0.195845
7	has_any_credit	-0.177416
6	balance_level	0.171812

Lojistik Regresyon modelinde en etkili 10 özelliğin katsayılarını (coefficient) gösteriyor.

Katsayı pozitifse $\rightarrow$ bu özelliğin artması, pozitif sınıfa (örneğin “abone oldu”) geçiş ihtimalini artırır.

Katsayı negatifse $\rightarrow$ bu özelliğin artması, pozitif sınıfa geçiş ihtimalini azaltır.

Özellikler

- `contact_unknown` = -0.54 = Eğer bireyle iletişim tipi “unknown” ise, abone olma olasılığı azalır.
- `poutcome_success` = +0.42 = Önceki kampanya sonucu “başarılı” olan bireylerin abone olma ihtimali artar.
- `month_aug` = -0.29 = Ağustos ayında yapılan görüşmelerde abone olma ihtimali daha düşüktür.
- `month_nov` = -0.25 = Kasım ayında abone olma olasılığı azalır.
- `month_may` = -0.25 = Mayıs ayında abone olma olasılığı da düşük.
- `month_jul` = -0.24 = Temmuz da yine negatif etkili bir ay.
- `campaign` = -0.22 = Kişiye yapılan arama sayısı arttıkça abone olma ihtimali düşer. (Belki rahatsız hissediyor.)
- `month_jan` = -0.19 = Ocak ayında abone olma ihtimali düşük.
- `has_any_credit` = -0.17 = Kredisi olan kişiler daha az abone oluyor olabilir. (Negatif ilişki)
- `balance_level` = +0.17 = Daha yüksek bakiye seviyesi, abone olma ihtimalini artırıyor.

Özellikle iletişim tipi, önceki kampanya sonucu ve ay gibi kategorik veriler önemli etkiye sahip.

`poutcome_success` gibi pozitif katsayılı özellikler → modelin “evet” demesinde etkili.

`contact_unknown`, `campaign` gibi negatif katsayılar → karar aleyhine etkili.

1.1 CV uygulanmış model sonuçları

```
[128]: # 1. Gerekli kütüphaneleri içe aktar
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np
```

```
[129]: # Pipeline: scaler + logistic regression birlikte
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression(max_iter=1000, solver='liblinear',
    ↪class_weight='balanced'))
])

# Stratified K-Fold: Sınıf dağılımı dengelenmiş 5 katlı CV
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# ROC AUC skoru ile çapraz doğrulama
scores = cross_val_score(pipeline, X, y, cv=cv, scoring='roc_auc')

print("Cross-validation ROC AUC skorları:", scores)
print("Ortalama ROC AUC:", np.mean(scores))
```

Cross-validation ROC AUC skorları: [0.76508269 0.77789594 0.75798573 0.76486225

0.77654594]

Ortalama ROC AUC: 0.7684745087450469

- Pipeline: Veri ön işleme ve modelleme işlemlerini tek adımda yapar.
- StandardScaler(): Verilerin standartlaştırılmasını sağlar (ortalama = 0, std = 1).
- LogisticRegression(): Lojistik regresyon modeli; class_weight='balanced' ile sınıf dengesizliği varsa bunu telafi eder. solver='liblinear', küçük veri kümelerinde iyidir.
- cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
 - Verisetini 5 parçaya böler.
 - Her katta sınıf oranlarını korur (örneğin pozitif-negatif örnek oranı her katta benzer olur).
- scores = cross_val_score(pipeline, X, y, cv=cv, scoring='roc_auc')
 - cross_val_score: Pipeline'ı 5 kez farklı veri seti bölmeleri üzerinde çalıştırır.
 - scoring='roc_auc': Her kat için ROC AUC skoru hesaplanır. Bu metrik, modelin sınıfları ne kadar iyi ayırdığını gösterir (1.0 = mükemmel, 0.5 = rastgele tahmin).

Sonuçların Yorumlanması:

Cross-validation ROC AUC skorları: [0.76508269 0.77789594 0.75798573 0.76486225 0.77654594]

Ortalama ROC AUC: 0.7684745087450469

- ROC AUC skorları 0.76 - 0.78 arasında değişiyor → Bu, modelin sınıfları ayırma becerisinin iyi olduğunu gösteriyor.
- Ortalama ROC AUC 0.768 → Genelde kabul edilebilir bir performanstır. Özellikle veriler arasında dengesizlik varsa, bu skor dengeli sınıflandırmada başarılı olduğunu gösterir.

```
[130]: # Tüm veriyle final model eğitimi (ölçekleme dahil)
pipeline.fit(X, y)
```

```
[130]: Pipeline(steps=[('scaler', StandardScaler()),
                      ('model',
                       LogisticRegression(class_weight='balanced', max_iter=1000,
                                           solver='liblinear'))])
```

- Pipeline, önce veriyi ölçeklendiriyor (StandardScaler), ardından Lojistik Regresyon modeli ile eğitiyor.
- Burada X ve y, tüm veriyi temsil ediyor.
- Yani artık final model tüm veriyle eğitildi.

```
[131]: # Doğrudan X_test verisini veriyoruz - scaler pipeline içinde zaten var
y_pred = pipeline.predict(X_test)

# Sonuçlar
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```

[[9265 2701]
 [ 613  985]]
      precision    recall  f1-score   support

     0       0.94      0.77      0.85     11966
     1       0.27      0.62      0.37      1598

 accuracy                   0.76     13564
 macro avg       0.60      0.70      0.61     13564
weighted avg       0.86      0.76      0.79     13564

```

- Eğitilen model, X_test (test verisi) için tahmin yapıyor.
- Önemli not: X_test verisi de pipeline içindeki scaler sayesinde otomatik olarak ölçekleniyor.
- Bu çıktılar sayesinde doğru ve yanlış sınıflandırmaların dağılımı ve Precision, Recall, F1-score, Accuracy gibi metrikler görüntüleniyor.
- 9265: Gerçek sınıf 0 ve doğru tahmin edilmiş (True Negative)
- 2701: Gerçek sınıf 0 ama yanlışlıkla 1 tahmin edilmiş (False Positive)
- 613: Gerçek sınıf 1 ama yanlışlıkla 0 tahmin edilmiş (False Negative)
- 985: Gerçek sınıf 1 ve doğru tahmin edilmiş (True Positive)

Accuracy (Doğruluk): %76 → Tüm tahminlerin %76'sı doğru.

Macro Avg (Sınıflar arası ortalama):

Precision: 0.60

Recall: 0.70

F1-score: 0.61 → Sınıflar arasında dengeye bakıldığında modelin pozitif sınıfta ciddi zayıflığı var.

Weighted Avg (Destek sayısına göre ağırlıklı ortalama):

Precision: 0.86

Recall: 0.76

F1-score: 0.79 → Sınıf 0 sayısı çok fazla olduğu için genel ortalamaları yukarı çekmiş.

- Güçlü Yönler:

Negatif sınıf (0) çok iyi tanımlanmış: yüksek precision ve f1-score.

Genel doğruluk fena değil (%76).

- Zayıf Yönler:

Pozitif sınıfta (1) precision çok düşük (%27): Model, 1 sınıfını tahmin ettiğinde sık sık yanıltıyor.

Bu dengesizlik genellikle sınıf dağılımının dengesiz olması (yani “class imbalance”) nedeniyle olur. (0: 11966, 1: 1598 → ciddi bir fark var.)

```
[132]: # Örneğin Roc_Curve skoru:
ruc_scores = cross_val_score(pipeline, X, y, cv=cv, scoring='roc_auc')
print("Roc_Curve skorları:", ruc_scores)

# Örneğin F1 skoru:
f1_scores = cross_val_score(pipeline, X, y, cv=cv, scoring='f1')
print("F1 skorları:", f1_scores)

# Örneğin Accuracy skoru:
accuracy_scores = cross_val_score(pipeline, X, y, cv=cv, scoring='accuracy')
print("Accuracy skorları:", accuracy_scores)
```

Roc_Curve skorları: [0.76508269 0.77789594 0.75798573 0.76486225 0.77654594]

F1 skorları: [0.37585812 0.38234467 0.36449115 0.37370838 0.37577465]

Accuracy skorları: [0.75870839 0.75702278 0.74585269 0.75868171 0.75492148]

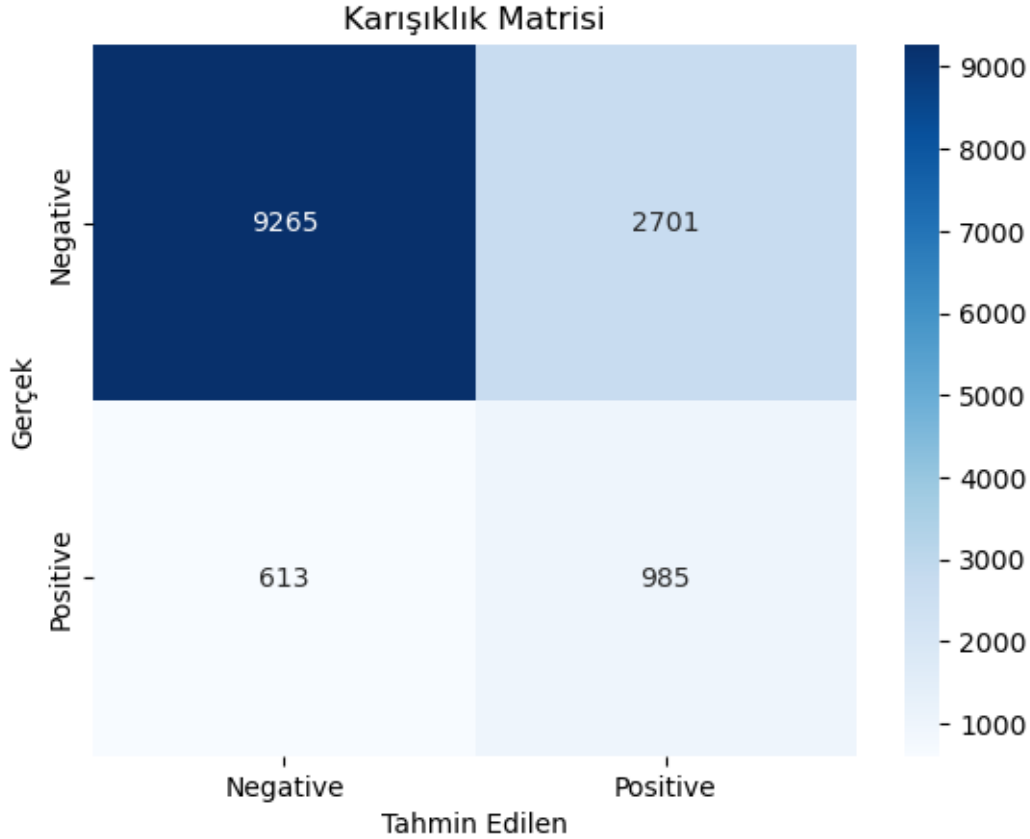
- 5 katlı Stratified K-Fold çapraz doğrulama ile bu metriklerin her kat için değeri hesaplanıyor. Bu, modelin genelleme yeteneğini test etmek için çok önemli bir yöntem.

Roc_Curve_Score : Modelin pozitif ve negatif sınıfları ayırma başarısı iyi düzeyde. ROC AUC > 0.75 genellikle tatmin edici kabul edilir.

F1 Skorları: F1 skoru düşüktür → bu, özellikle pozitif sınıfın tahmininde zorlanma yaşandığını gösterir. Muhtemelen pozitif sınıf çok az sayıda olabilir (sınıf dengesizliğinden kaynaklı olabilir).

Accuracy Skorları: Genel doğruluk da fena değil, ama bu değer dengesiz veri setlerinde yanıltıcı olabilir. Örneğin pozitif örnekler çok azsa, model hep negatif diyerek yüksek accuracy alabilir ama işe yaramaz.

```
[133]: # 9. Karışıklık matrisi
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Negative', 'Positive'], yticklabels=['Negative', 'Positive'])
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek')
plt.title('Karışıklık Matrisi')
plt.show()
```

True Negative (TN): 9265 Gerçek değeri negatif olan 9265 örneği doğru bir şekilde negatif olarak tahmin etmiş.

False Positive (FP): 2701 Gerçek değeri negatif olan ama pozitif olarak tahmin edilen 2701 örnek var. Bu durum “yanlış alarm” olarak adlandırılır.

False Negative (FN): 613 Gerçek değeri pozitif olan ama negatif tahmin edilen 613 örnek var. Bu durumda pozitif bir olayı gözden kaçırmış oluyor (örneğin bir hastalığı atlamak gibi kritik olabilir).

True Positive (TP): 985 Gerçekten pozitif olan 985 örnek doğru şekilde pozitif olarak tahmin edilmiş.

Modelin Başarısı: - Negatif sınıf (majority class):

Oldukça yüksek doğrulukta tahmin edilmiş. TN sayısı yüksek (9265), FP sayısı da makul (2701).

- Pozitif sınıf (minority class):

TP sayısı 985 olsa da FN sayısı 613, yani pozitifleri ayırt etmekte zorlanıyor.

Aynı zamanda FP sayısı yüksek (2701), bu da precision (kesinlik) değerini düşürüyor.